RLHF

What is Preference Learning or Preference Alignment or Preference Tuning?
Why do we need that?
What is a problem with only SFT?
All the methods for the Preference Alignment?
RL(Reinforcement Learning) Introduction
RL Evolution Timeline
Diff B/W SL & RL

PART-1

Q-Learning v/s DQN(Deep Q-Network) v/s PPO
Practical of RL

PART-2

What is RLHF?
PPO in RLHF
Practical of RLHF with Custom dataset

PART-3

Before understanding preference tuning or any of its methods, let's first understand the LLM training pipeline and where preference tuning fits within it.

Pretraining
(Train the model on a massive dataset to learn general knowledge and language patterns)
↓
Supervised Fine-Tuning (Instruction Tuning)
(Instruction-based tuning to teach the model how to follow user instructions)
↓
Preference Tuning / Preference Alignment(Behavior control)
(Preference-based tuning to align model outputs with human preferences)
Methods: RLHF (PPO), GRPO, DPO, ORPO

What all methods are there for this?

Method	Uses RL?	Reward Model?	Complexity
PPO	✅	✅	Very High
GRPO	✅	✅	High
DPO	❌	❌ (*implicit preference objective*)	Medium
ORPO	❌	❌	Low

RLHF ← OpenAI

Deepseek

RLHF(PPO) and GRPO
- Complex
- Expensive
- Multiple models needed: Base model , Reward model , RL training loop

DPO (MODERN ALTERNATIVE)
Directly train model:
Chosen > Rejected

ORPO (LATEST METHOD)
SFT + Preference learning in one loss

What is preference alignment?

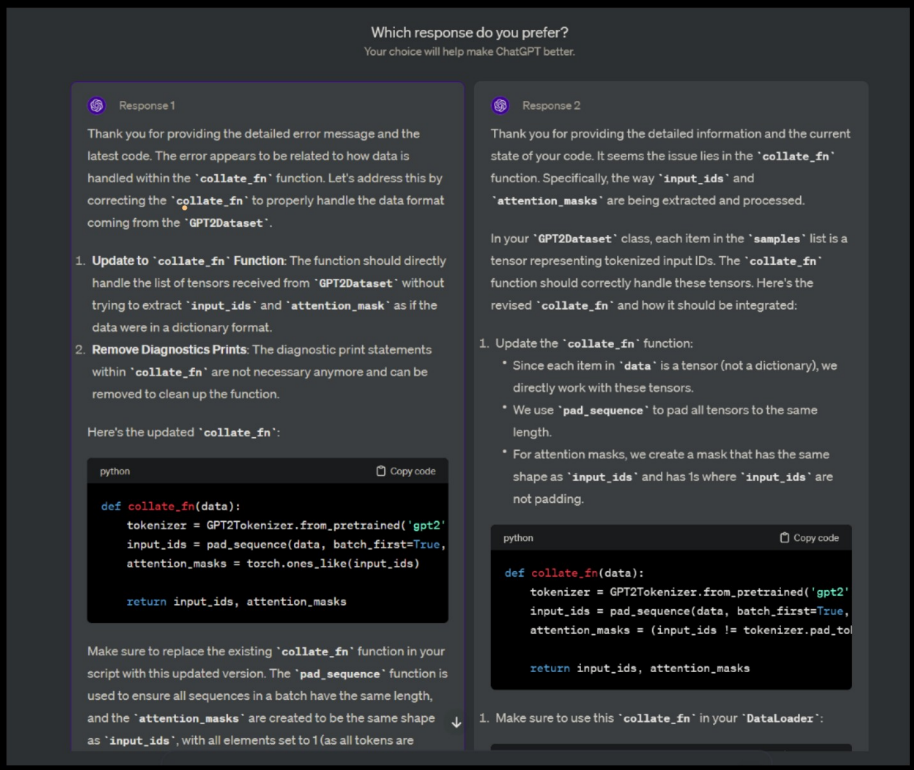
Preference alignment is the process of training a model to generate outputs that humans prefer.

In simple terms, among many possible answers, it helps the model learn which response humans like more.

Example
Prompt: "How do I prepare for exams?"

Possible Outputs:
First Response: "Make a study plan, revise daily, and take proper breaks to stay consistent."
Second Response: "If you're not studying 10 hours a day, you're going to fail."

✅ Calm, structured advice → Preferred
❌ Overly harsh, unsafe advice → Rejected



Why do we need Preference Alignment?
Problem with only SFT (Supervised Fine-Tuning)

Because Supervised Fine-Tuning (SFT) improves task performance

But it does not ensure that the model's behavior aligns with human values, safety, and expectations.

Alignment techniques like RLHF, DPO, and ORPO ensure that LLMs not only generate correct outputs but also produce responses that are safe, helpful, and aligned with human preferences.

Preference alignment (also called preference learning) is now a critical part of modern LLM training—it is no longer optional.

Modern LLMs are used in real-world systems such as:

- Healthcare assistants
- Finance tools
- Education platforms
- Enterprise copilots

In these environments, behavior matters just as much as correctness.

Pretraining means predict the next token

Pretrained models are good at:

- Generating fluent text

In Pretraining Model learns:

- Patterns in data
- Grammar & language structure
- General knowledge

But the training data may contain:

- Incorrect or misleading information
- Not following instructions properly
- Not Giving consistently correct or helpful answers

"Pretraining makes the model knowledgeable, but not controllable and trustworthy"

Supervised Fine-Tuning teaches the model:

- How to follow instructions
- The expected format of answers

SFT teaches:

- "This is a good answer"

But it does NOT explicitly say:

- "This answer is bad, avoid it"

What SFT does not guarantee

SFT does not explicitly teach:

- Which answers are better than others
- Which answers are unacceptable
- Human values like politeness, safety, tone, or helpfulness

So during SFT:

- Probability of good answers increases
- Probability of bad answers can also increase accidentally

Assume you are training a model using Supervised Fine-Tuning (SFT).

Training Data (SFT)



User:
"What is the capital of France?"

Model is taught:
"Paris is the capital of France."

That is all the model sees — only the correct answer.

Where is the problem?

Now in the real world:

User:
"What is the capital of Germany?"

The model could generate two possible outputs:

"Paris is the capital of Germany." (incorrect)
"Berlin is the capital of Germany." (correct)
What does SFT actually do?

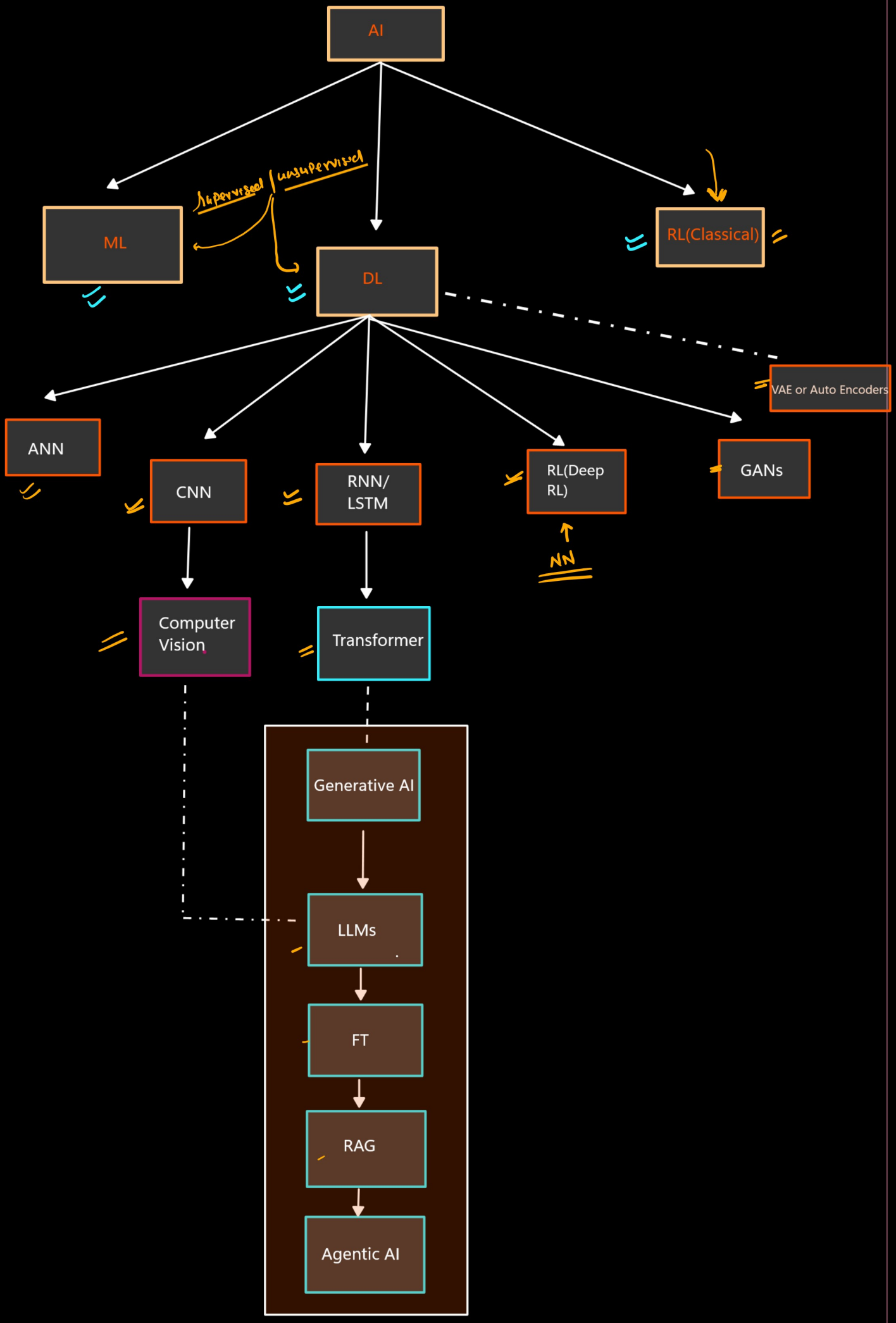
SFT only teaches:
"Paris is correct for France."

But it does NOT teach:
"Paris is wrong for Germany."

preference alignment ensure the model is:

- ✅ Helpful
- ✅ Harmless
- ✅ Honest
- ✅ Polite
- ✅ Safe
- ✅ Non-toxicity
- ✅ Consistency with human expectations

In short:
Preference alignment bridges the gap between “knows language” and “behaves well”.


AI
ML
Supervised
unsupervised
DL
RL(Classical)
VAE or Auto Encoders
ANN
CNN
RNN/
LSTM
RL(Deep
RL)
GANs
NN
Computer
Vision
Transformer
Generative AI
LLMs
FT
RAG
Agentic AI

What is RL?

It is a way of building intelligent systems(AI systems).

Reinforcement Learning (RL) is a learning method where a model learns through trial and error.

An agent learns by interacting with its environment and receiving rewards or penalties.

Over time, it learns to choose the best possible actions.

One Line Definition: "Reinforcement Learning is about learning optimal behavior through rewards and penalties."
"Take action → get feedback → improve decision over time"

What is Reinforcement Learning (RL)?

Reinforcement Learning is a type of Machine Learning where an agent learns by interacting with an environment and receiving rewards or penalties for its actions.

1. Agent
The learner (model)

2. Environment
Where the agent acts (game, chatbot, etc.)

3. Action
What the agent does

4. State
Current situation

5. Reward
Feedback (good or bad)

The whole idea of:
State → Action → Environment → Reward → Update → Repeat
can be applied anywhere.

Simple Example (Game): Imagine a game

- Agent = Player
- Action = Move left/right
- Reward:
  - +10 → win
  - -10 → lose



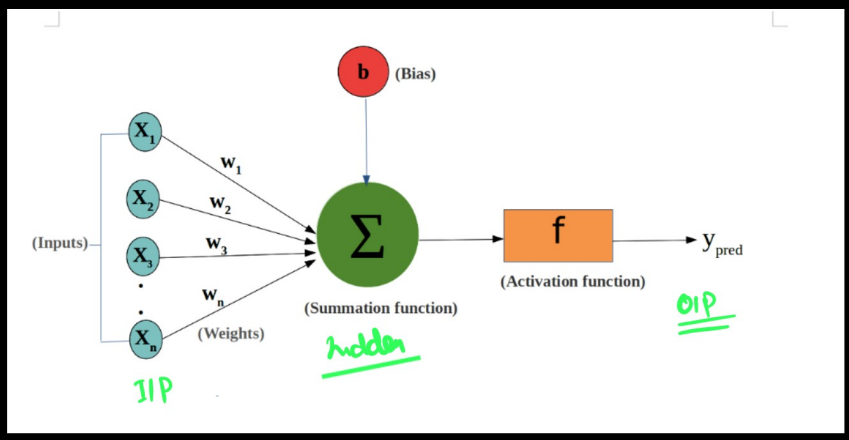
This example explains Reinforcement Learning using a simple real-life analogy.

Will understand it programmatically with a coding example also.

RL Evolution Timeline

RL evolved from tables → neural networks → human-aligned intelligence."

Phase	Description
1. 1950s – Foundations (MDP)	• Reinforcement Learning is based on Markov Decision Process (MDP) • Core idea: State → Action → Reward → Next State This is still the backbone of RL today
2. 1980s–1990s – Classical RL	• Key algorithms introduced: ○ Q-Learning ○ Temporal Difference (TD Learning) ✔ Learning using tables Not scalable to large problems
3. 2000s – Slow Progress	• Mostly theoretical work • Limited real-world applications Reason: • Low compute power • No deep learning support
4. 2013–2015 – Deep Reinforcement Learning (Breakthrough)	• Introduction of Deep Q-Network (DQN) ✔ Neural networks + RL ✔ Solved Atari games This is where RL became practical
5. 2016 – Policy Gradient & Actor-Critic	• Shift from value-based → policy-based learning ✔ Instead of Q-values, we directly learn: $\pi(a \mid s)$
6. 2017 – PPO (Most Important Phase)	Proximal Policy Optimization (PPO) introduced ✔ Stable training ✔ Controlled updates ✔ Widely used in industry This is the core RL algorithm used in many systems
7. 2018–2020 – RL in Real Systems	• Used in: ○ Robotics ○ Game AI (AlphaGo, AlphaZero) ✔ Complex decision-making tasks
8. 2020–2022 – RL + LLM (RLHF)	• Reinforcement Learning applied to LLMs RLHF (Reinforcement Learning with Human Feedback) Pipeline: Pretraining → SFT → Reward Model → PPO ✔ Used to train ChatGPT-like models



Q-learning ← tabular

NN

What is Deep RL?

Deep RL → RL + neural networks

Deep Learning learns patterns from data using neural networks, Reinforcement Learning learns optimal actions using reward signals and Deep Reinforcement Learning combines both to solve complex decision-making problems using neural networks.

Deep RL → RL powered by neural networks

AlphaGo (Google)

ChatGPT (Open AI)

RL → PPO ← NN

~~SL~~

State → NN → Action

Action | Reward | Reality

~~Agent~~

Concept	Supervised Learning	Reinforcement Learning
Data	Labeled data	No labels
Feedback	Immediate (correct/incorrect)	Delayed reward
Goal	Predict correct output	Maximize reward
Learning Style	Direct learning	Trial & Error
Example	Image classification	Game playing / ChatGPT alignment

O/P

Supervised Learning = Teacher present

RL = No teacher, only reward system

SL

I/P O/P

w/b

NN → O/P ↔ Actual

Optimizer

BP

Loss

What is "learning" in RL

Forget theory. Think like this:

Learning = changing behaviour based on feedback

What actually changes?

In ML / DL: Learning = updating parameters (weights)

In RL

Model has: Policy π(a | s)

Meaning: Probability of taking action "a" in state "s"

Learning = change this probability:

Good action → probability ↑

Bad action → probability ↓

Difference between Supervised Learning and Reinforcement Learning

"Supervised learning optimizes against correct answers, while reinforcement learning optimizes based on rewards without knowing the correct answer."

1. Supervised Learning (Simple Form)
Objective (Simple)
"Loss"="Average Error between Prediction and True Answer"
Update Rule (Simple)
θ=θ−"learning rate"×"gradient"
Meaning:

- Agar error zyada hai → weights change karo
- Goal = prediction ≈ correct answer

2 Reinforcement Learning (Simple Form)
Objective (Simple)
Goal:

"Maximize Total Reward"

$$J(\theta) = \mathbb{E}[r_1 + r_2 + r_3 + \dots]$$

Symbol	Meaning
s	State (current situation)
a	Action (decision taken)
$\pi_\theta(a \mid s)$	Probability of taking action a in state s
r	Reward / feedback
θ	Model parameters
$J(\theta)$	Total expected reward

Goal:

"Increase probability of good actions"

Simple intuition:

Update∝Reward×log(Probability of action)
Meaning
If reward is high:

✅ Increase probability of taking that action again.

If reward is low:

❌ Reduce probability of that action.

Difference between Deep Learning and Reinforcement Learning(Deep RL)

Concept	Classical RL	Deep Learning (DL)	Deep Reinforcement Learning (Deep RL)
Input	State (discrete)	Input data (image, text, etc.)	State (high-dimensional, continuous)
Output	Action	Prediction	Action
Core Idea	Q-table / Policy table	Neural network learning	RL + Neural Network
Feedback Signal	Reward	Loss	Reward-based loss
Objective	Maximize cumulative reward	Minimize loss	Maximize reward
Learning Type	Trial & Error	Supervised / Unsupervised	Trial & Error + NN
Representation	Tabular (Q-table)	Neural Network	Neural Network (Q(s,a), Policy)
Training Process	Update Q-table	Forward → Loss → Backprop	Forward → Reward → Loss → Backprop
Algorithms	Q-learning, SARSA	CNN, RNN, Transformers	DQN, PPO
Data Requirement	Environment interaction	Dataset (labeled/unlabeled)	Environment interaction
Use Case	Simple games, small problems	Vision, NLP	Robotics, Games, LLM alignment

Reinforcement Learning is a learning method where an agent learns by interacting with an environment and receiving rewards or penalties for its actions.

Agent = Self-driving car
Action = Brake / accelerate / turn
Environment = Road traffic
Reward = Safe driving

Simple Example (Game): Imagine a game

- Agent = Player
- Action = Move left/right
- Reward:
  - +10 → win
  - -10 → lose

Simple Real-Life Example

Suppose a baby is learning walking.
Baby walks correctly → parents clap → positive reward
Baby falls → pain → negative reward

Slowly baby learns:
Which actions are good
Which actions are bad

This is Reinforcement Learning.

Componant of RL.

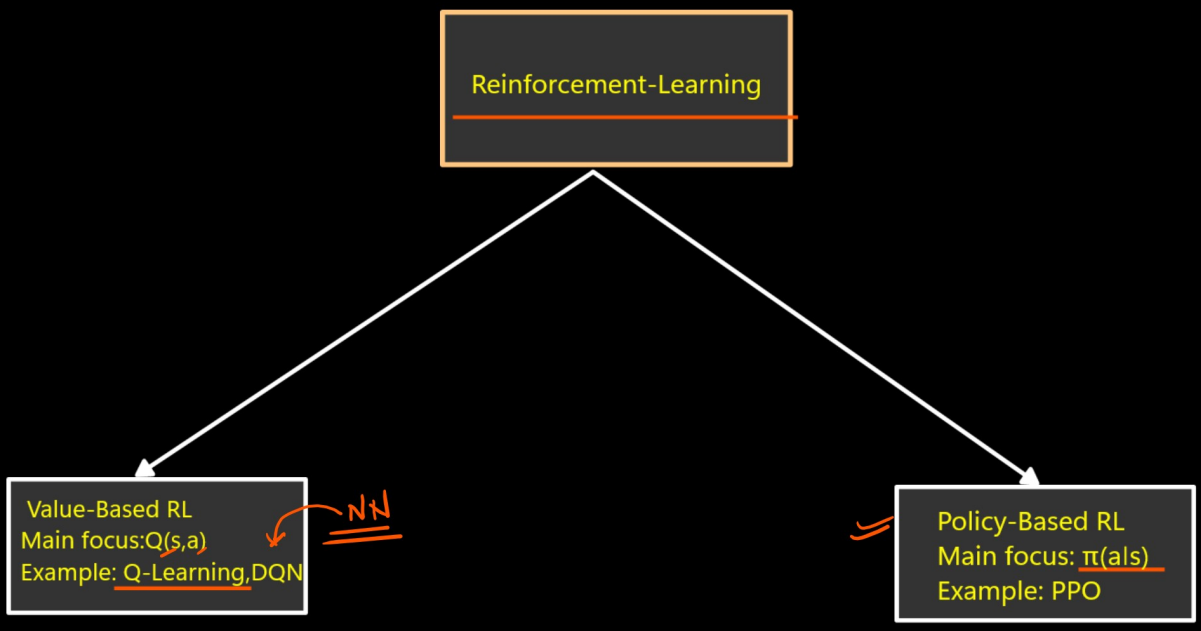
1. Agent
The learner (model)

2. Environment
Where the agent acts (game, chatbot, etc.)

3. Action
What the agent does

4. State
Current situation

5. Reward
Feedback (good or bad)


Reinforcement-Learning
Value-Based RL
Main focus:Q(s,a)
Example: Q-Learning,DQN
NN
Policy-Based RL
Main focus: π(a|s)
Example: PPO

RL Evolution Timeline

RL evolved from tables → neural networks → human-aligned intelligence."

Phase	Description
1. 1950s – Foundations (MDP)	• Reinforcement Learning is based on Markov Decision Process (MDP) • Core idea: State → Action → Reward → Next State This is still the backbone of RL today
2. 1980s–1990s – Classical RL	• Key algorithms introduced: ○ Q-Learning ○ Temporal Difference (TD Learning) ✔ Learning using tables Not scalable to large problems
3. 2000s – Slow Progress	• Mostly theoretical work • Limited real-world applications Reason: • Low compute power • No deep learning support
4. 2013–2015 – Deep Reinforcement Learning (Breakthrough)	• Introduction of Deep Q-Network (DQN) ✔ Neural networks + RL ✔ Solved Atari games This is where RL became practical
5. 2016 – Policy Gradient & Actor-Critic	• Shift from value-based → policy-based learning ✔ Instead of Q-values, we directly learn: $\pi(a \mid s)$
6. 2017 – PPO (Most Important Phase)	• Proximal Policy Optimization (PPO) introduced ✔ Stable training ✔ Controlled updates ✔ Widely used in industry This is the core RL algorithm used in many systems
7. 2018–2020 – RL in Real Systems	• Used in: ○ Robotics ○ Game AI (AlphaGo, AlphaZero) ✔ Complex decision-making tasks
8. 2020–2022 – RL + LLM (RLHF)	• Reinforcement Learning applied to LLMs RLHF (Reinforcement Learning with Human Feedback) Pipeline: Pretraining → SFT → Reward Model → PPO ✔ Used to train ChatGPT-like models

1. Q(Quality)-Learning

Type: Value-Based
Storage: Table

Stores:
Q(s, a) in a lookup table

Works well for:
Small / discrete state spaces

Problem:
State space becomes large → table size explodes

2. DQN (Deep Q-Network)
Type: Value-Based
Upgrade of: Q-Learning(tabluar)

Key Idea:
Replace Q-table with Neural Network

Input: State
Output: values for all actions

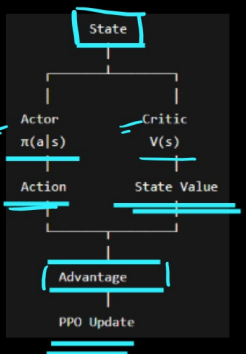
Solves:
Large state space (images, games, high-dimensional input)

Limitation:
Works mainly for discrete action spaces with continues values training can be unstable

3. PPO (Proximal Policy Optimization)
Type: Policy-Based (Actor-Critic)

Meaning: probability of taking action a given state s

Advantages:
Works with continuous actions (robotics, control systems)
Stable training (compared to vanilla policy gradient)
Widely used in robotics, games, and RLHF (e.g., ChatGPT training)
PPO is not purely policy-based
It is Actor-Critic (learns both policy and value function)



What is Q-learning?
Q-Learning is a value-based Reinforcement Learning algorithm

Q-Learning learns "how good" each action is in a particular state. It stores these values in a Q-Table and chooses the action with the highest Q-value.

Treasure Hunt Game
Current State
You are standing at a crossroad.
Available Actions
Go Left
Go Right
Go Straight
Goal
Find the treasure.



Value-Based RL says:
"Let's learn how good each action is in a given state."
State = Crossroad
Go Left = +10 points
Go Right = +50 points
Go Straight = +20 points
The agent learns and stores these values in a table called a Q-Table.

How Does the Agent Choose an Action?
Simple.
The agent looks at all Q-values for the current state and selects the action with the highest Q-value.
Go Left = 10
Go Right = 50
Go Straight = 20
Maximum Q-Value: 50
Corresponding Action:Go Right
Therefore, the agent chooses: Action = Go Right
What is Q(s,a)?
The Q-value represents:
"If I am in state s and take action a, how much total future reward can I expect to receive?"
Mathematically:Q(s,a)
Where:
s = State
a = Action

Example

For our crossroad example:

Q(Crossroad, Go Left) = 10

Q(Crossroad, Go Right) = 50

Q(Crossroad, Go Straight) = 20

This means:

Going Left is expected to give a future reward of 10.
Going Right is expected to give a future reward of 50.
Going Straight is expected to give a future reward of 20.

Q-Table

State	Action	Q-Value
Crossroad	Go Left	10
Crossroad	Go Right	50
Crossroad	Go Straight	20

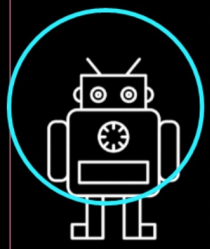
Q-Table Explosion Problem
For small problems, Q-learning works fine:

```
Q = {
   "left": 0,
   "right": 0
}
```

This is manageable.

Now think:
State = image (car camera)
1 image = 1000 × 1000 pixels
Possible states = billions

Q-table issue
• A row is needed for every state
• A column is needed for every action
Impossible to store



Q	a1	a2	a3	a4
S1	Q(S1,a1)	Q(S1, a2)	Q(S1, a3)	Q(S1, a4)
S2	Q(S2,a1)	Q(S2,a2)	Q(S2, a3)	Q(S2, a4)
S3	Q(S3,a1)	Q(S3,a2)	Q(S3, a3)	Q(S3, a4)
.	.	.	.	.

Neural Network entry (DQN)

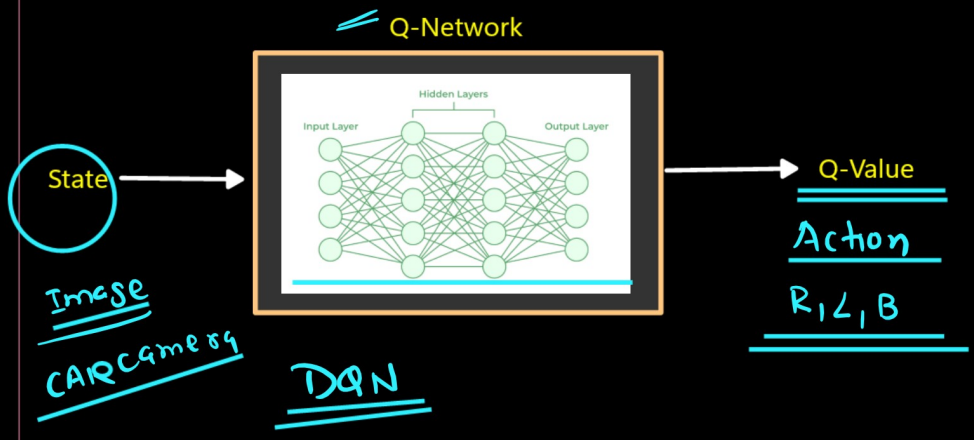
Instead of creating a table: "We have a function(Neural Network) that predicts Q values"

Q-table lookup[Q(s,a)] = Neural Network(s)

Example
Input: Image (state)
Output: Q values for actions

Input: Car camera image
Output:
Left = 2.1
Right = 5.6
Brake = 3.0

Q-Learning	DQN
Stores table	Learns function
Memory heavy	Generalizes
Exact values	Approximate values



State S1

↓

Q-Network predicts Q-values

↓

Choose action Right

↓

Environment gives reward +5 and next state S2

↓

Store (S1, Right, +5, S2) ←

↓

During training, sample this experience

↓

Q-Network predicts current value Q(S1, Right)

↓

Target Network predicts future value from S2

↓

Target = reward + gamma × best future value ← learning

↓

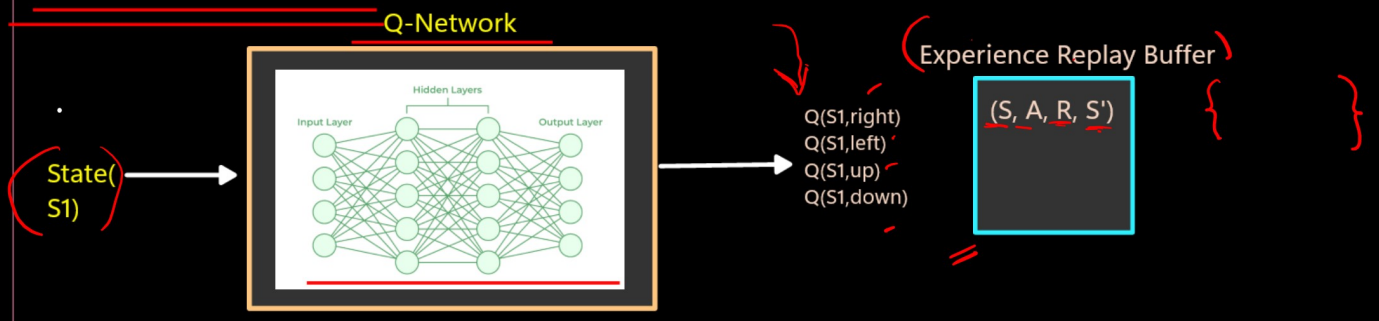
Loss = target - current prediction

↓

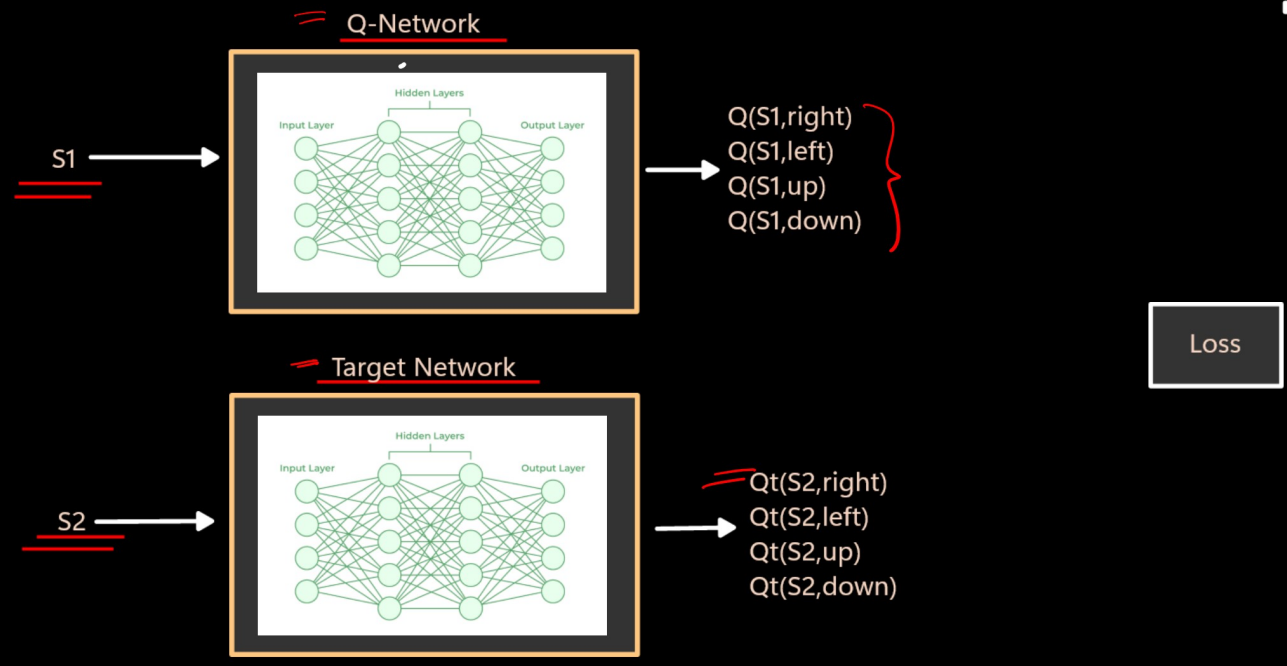
Update Q-Network

DQN → 2 phase

Data Prepration Phase



Training Phase

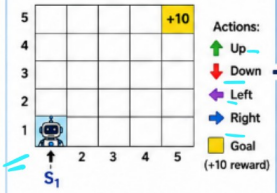


DQN is not supervised learning because it does not learn from labeled; instead, it learns through trial-and-error interactions with the environment using rewards as feedback.

DQN (Deep Q-Network) Example – Step by Step

1 Step 1: Robot current state

State = S_1



Actions:
- Up
- Down
- Left
- Right

Goal (+10 reward)

2 Step 2: Q-Network prediction

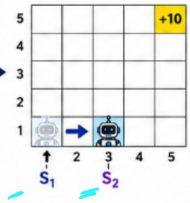
Input: S_1

Action	$Q(S_1, \cdot)$
Left	1
Right	4
Up	2
Down	0

Right action has the highest value.

Robot chooses: Action = Right

3 Step 3: Environment gives reward



Reward = +5

Next State = S_2

Stored in Replay Buffer: (S_1, Right, +5, S_2)

4 Step 4: During training, use the same data

Sample from Replay Buffer: (S_1, Right, +5, S_2)

DQN needs to learn: What should be the correct value of taking Right action in state S_1?

5 Step 5: Q-Network current prediction

Input: S_1

Action	$Q(S_1, \cdot)$
Left	1
Right	4
Up	2
Down	0

Current Prediction: $Q(S_1, \text{Right}) = 4$

6 Step 6: Target Network looks into the future

Input: S_2

Action	$Q_{target}(S_2, \cdot)$
Left	3
Right	6
Up	2
Down	1

Best future value: max = 6

7 Step 7: Calculate target value

Target = Reward + γ × Best Future Value

= 5 + 0.9 × 6

= 5 + 5.4

= 10.4

Meaning: The correct value of taking Right action in state S_1 should be approximately 10.4.

8 Step 8: Compare prediction vs target

Current Prediction $Q(S_1, \text{Right}) = 4$

VS

Target = 10.4

Error = 10.4 – 4 = 6.4 (Underestimated)

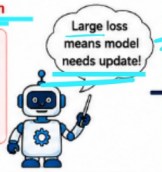
9 Step 9: Loss calculation

Loss = (Target – Prediction)²

= (10.4 – 4)²

= 6.4²

= 40.96



10 Step 10: Q-Network update (after training, values improve)

Before Update

Action	$Q(S_1, \cdot)$
Left	1
Right	4
Up	2
Down	0

After some training

Action	$Q(S_1, \cdot)$
Left	1
Right	7
Up	2
Down	0

More training

Action	$Q(S_1, \cdot)$
Left	1
Right	9
Up	2
Down	0

Eventually

Action	$Q(S_1, \cdot)$
Left	1
Right	10.4
Up	2
Down	0

$Q(S_1, \text{Right}) \approx 10.4$ (Correct value ke close)

Example Game: The robot is in a grid world.

Actions: Left, Right, Up, Down

Goal: Reach the box with a reward of +10.

Step 1: The robot is in the current state
State = S1

Step 2: The Q-Network makes a prediction

The Q-network receives S1 as input.

The Q-network outputs:

Q(S1, Left) = 1
Q(S1, Right) = 4
Q(S1, Up)   = 2
Q(S1, Down) = 0

Meaning: The Right action seems to be the best action.

Because: Q(S1, Right) = 4 is the highest value.

The robot chooses:

Action = Right

Step 3: The environment gives a reward

The robot moves Right.

The environment returns:

Reward = +5

Next State = S2

The following experience is stored in the Replay Buffer:

(S1, Right, +5, S2)

Step 4: During training, we use the same stored data

A sample is taken from the Replay Buffer:

(S1, Right, +5, S2)

Now DQN needs to learn:

"What should be the correct value of taking the Right action in state S1?"

Step 5: The Q-Network gives the current prediction

The Q-network looks at S1 again:

Q(S1, Left) = 1
Q(S1, Right) = 4
Q(S1, Up)   = 2
Q(S1, Down) = 0

Since the action taken was Right, we only use:

Current Prediction = Q(S1, Right) = 4

Step 6: The Target Network looks into the future

The Target Network receives the next state:

S2

The Target Network outputs:

Q_target(S2, Left) = 3
Q_target(S2, Right) = 6
Q_target(S2, Up)   = 2
Q_target(S2, Down) = 1

Best future value:

max = 6

Meaning:

If the agent takes the best possible action from S2, it expects to receive a future reward of 6.

Step 7: Calculate the Target Value

Formula:

Target = Reward + gamma × Best Future Value

Suppose:

Reward = +5
gamma = 0.9
Best Future Value = 6

Then:

Target = 5 + 0.9 × 6
Target = 5 + 5.4
Target = 10.4

Meaning:

The correct value of taking the Right action in state S1 should be approximately 10.4.

Step 8: Compare Prediction vs Target

The Q-network predicted:

Q(S1, Right) = 4

The target value says:

Correct Value = 10.4

Error:

10.4 - 4 = 6.4

This means the Q-network underestimated the value of taking the Right action.

Step 9: Compute the Loss

Loss = (Target - Prediction)²

Loss = (10.4 - 4)²

Loss = 6.4²

Loss = 40.96

Step 10: Update the Q-Network

After training, the Q-network improves its prediction.

Before Training:

Q(S1, Right) = 4

After Some Training:

Q(S1, Right) = 7

After More Training:

Q(S1, Right) = 9

Eventually:

Q(S1, Right) ≈ 10.4

The Q-network gradually adjusts its weights so that its predicted
Q-value gets closer and closer to the target value.

Over time:

Prediction → Target

4 → 7 → 9 → 10.4

DQN (Deep Q-Network) predicts a Q-value for every possible action and then selects the action with the highest Q-value.
works very well with Discrete Action Space

Actions:
- Left
- Right
- Jump
- Shoot

DQN Output:

Q(Left) = 10
Q(Right) = 50
Q(Jump) = 20
Q(Shoot) = 30

The agent simply chooses the action with the highest Q-value (Right).
Since there are only a few actions, the neural network only needs a few output nodes.

Continuous Action Space
In many real-world problems, actions are continuous rather than fixed.
Example: Self-driving car steering angle

0°
0.1°
0.11°
0.111°
...

There are effectively infinitely many possible steering angles.

Since DQN must estimate a Q-value for every possible action, it would need:

Q(0°)
Q(0.001°)
Q(0.002°)
...

for an infinite number of actions.
This is computationally impossible because the neural network cannot produce an infinite number of outputs.

Why not DQN for RLHF?

For a single word, there may be 100,000+ possible tokens.
For a sentence, the combinations become enormous.

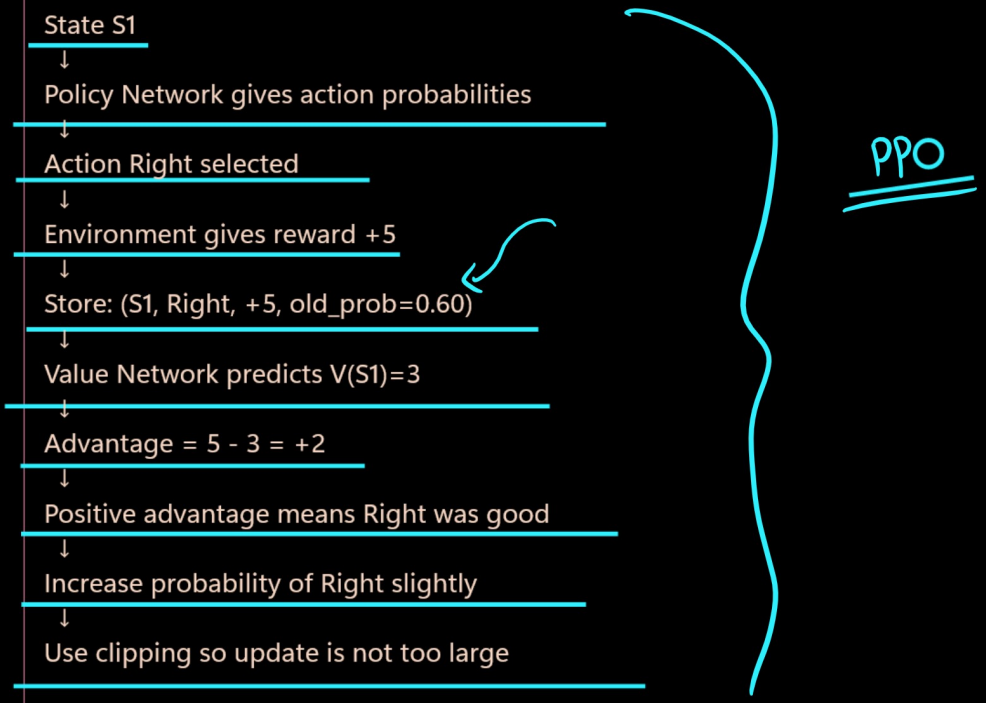
PPO ≠ DQN

DQN works well for discrete action spaces because the number of actions is finite. It struggles with continuous action spaces because it would need to estimate Q-values for infinitely many possible actions. Therefore, algorithms such as PPO, DDPG, TD3, and SAC are typically used for continuous control problems like robotics and self-driving cars.

RL

DQN asks:
"What is the Q-value of every action?"

PPO asks:
"How should I slightly adjust my behavior to get higher rewards?"

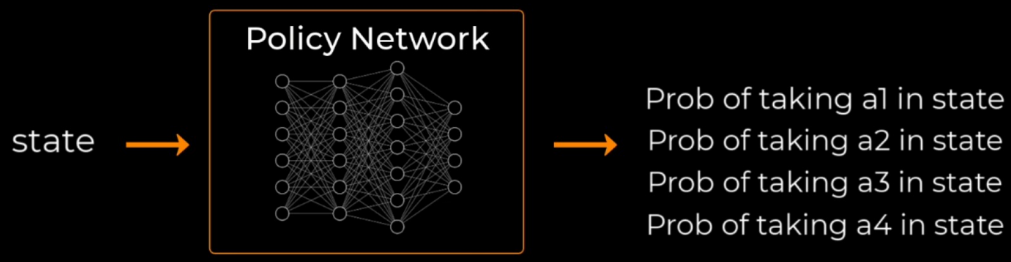


PPO (Proximal Policy Optimization) is a Reinforcement Learning algorithm that learns how to make better decisions over time.

Instead of learning Q-values like DQN, PPO directly learns a policy:

State → Best Action Probability

1. Policy Network (Actor)



Data Store

(State,
Action,
Reward,
Old Action Probability)

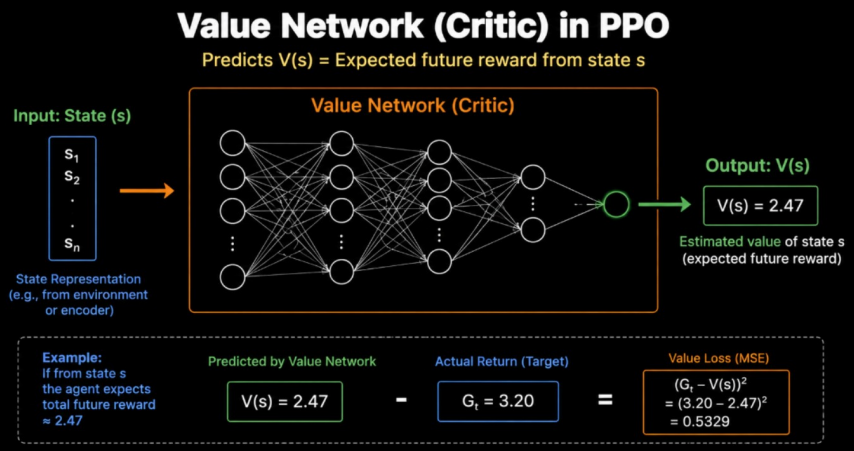
Proximal: The most important idea:
Don't change the policy too much at once.
Bad approach: 70% → 100%
Good PPO approach:b70% → 75%
Small safe updates.
This makes training stable.

Data Store

(State,
Action,
Reward,
Old Action Probability)

(S1, Right, -1, 0.51)
(S2, Down, +2, 0.70)
(S3, Left, +10, 0.85)

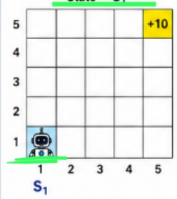
2. Value Network (Critic)



PPO (Proximal Policy Optimization) – Step by Step

1 Step 1: Policy Network makes a decision

State = S_1



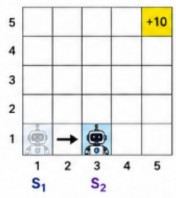
Policy Network Output

Action	Probability
Left	0.10
Right	0.60
Up	0.20
Down	0.10

Right has the highest probability.

Robot chooses: Action = Right
Store old probability of Right = 0.60

2 Step 2: Environment gives a reward



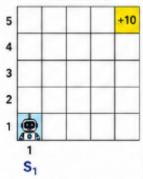
Reward = +5

Next State = S_2

Stored Data:
(S_1, Right, +5, old_prob = 0.60)

3 Step 3: Value Network makes a prediction

Value Network Input:
State = S_1



Value Network Output

$V(S_1) = 3$

This means:
The model expected a future reward of 3 from state S_1.

Actual Return obtained from environment = 5

4 Step 4: Advantage is calculated

Formula:
Advantage = Actual Return – Expected Value

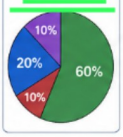
Advantage = 5 – 3
= +2

Meaning:
The action Right performed better than expected.

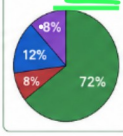
PPO decides:
Increase the probability of taking Right slightly.

5 Step 5: Policy is updated

Old Policy
P(Right) = 0.60



New Policy (after training)
P(Right) = 0.72



Ratio Calculation
ratio = new_prob / old_prob
= 0.72 / 0.60
= 1.2

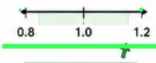
Meaning:
The action Right is now 20% more likely to be chosen.

PPO checks whether this update is too large.

6 Step 6: PPO Clipping

PPO main rule:
Do not change the policy too much at once.

Allowed Range (Clipping Range)



Our case:
ratio = 1.2
Allowed

If the update is too large:
Example:
P(Right) = 0.90
ratio = 0.90 / 0.60
= 1.5
Too Large
PPO will CLIP it to stay within the range.

Summary Flow

State (S_1) → Policy Network (decides action) → Take Action (Right) → Environment (gives reward (+5)) → Value Network (predicts value $V(S_1) = 3$) → Compute Advantage ($A = 5 - 3 = +2$) → Update Policy (P(Right): 0.60 → 0.72) → Compute Ratio ($\text{ratio} = \frac{\text{new}}{\text{old}} = 0.72 / 0.60 = 1.2$) → Apply Clipping (Keep ratio in [0.8, 1.2])

Example Game: The robot is in a grid world.

Actions: Left, Right, Up, Down

Goal: Reach the box with a reward of +10.

Step 1: Policy Network makes a decision

The robot is currently in state S1.

The policy network outputs:

P(Left) = 0.10
P(Right) = 0.60
P(Up) = 0.20
P(Down) = 0.10

This means: The robot thinks that moving Right has the highest probability.

The robot chooses: Action = Right

Important: Store the old probability of the chosen action.

Old probability of Right = 0.60

Step 2: Environment gives a reward

The robot moves Right.

The environment returns:

Reward = +5
Next State = S2

The following data is stored: (S1, Right, +5, old_prob = 0.60)

Step 3: Value Network makes a prediction

The job of the Value Network is to estimate:

"How much future reward can be obtained from this state?"

Suppose the Value Network predicts:

V(S1) = 3

This means:

The model expected a future reward of 3 from state S1.

However, the actual return obtained was:

Actual Return = 5

Step 4: Advantage is calculated

Formula: Advantage = Actual Return − Expected Value

Using the numbers: Advantage = 5 − 3 = +2

Meaning: The action Right performed better than expected.

Therefore, PPO decides: Increase the probability of taking Right slightly.

Step 5: Policy is updated

Old policy: P(Right) = 0.60

After training: P(Right) = 0.72

Now calculate the ratio:
ratio = new_prob / old_prob
ratio = 0.72 / 0.60 = 1.2

Meaning: The action Right is now 20% more likely to be chosen.
PPO checks whether this update is too large.

Step 6: PPO Clipping

The main rule of PPO is: Do not change the policy too much at once.

Suppose the allowed range is:
0.8 to 1.2

In our case: ratio = 1.2
This is allowed.
If the new probability had become:
P(Right) = 0.90

Then: ratio = 0.90 / 0.60 = 1.5

PPO would say: This update is too aggressive.

Clip it.

PPO overall training loop
1. Compute loss for Value Function Network
2. Compute loss for Policy Network
3. Update both networks
4. Repeat

Example: Training ChatGPT using RLHF + PPO

Step 1: User Prompt

State:

User: "What is Machine Learning?"

This is the current state for PPO.

Step 2: Policy Network Generates Response

The policy model, which is the LLM, gives token probabilities.

Example:

Response A:

"Machine Learning is a field of AI that enables systems to learn from data."

Suppose according to the policy model:

Probability(Response A) = 0.60

Store:

old_prob = 0.60

Step 3: Reward Model Scores the Response

The Reward Model evaluates it.

Prompt:

What is Machine Learning?

Response:

Machine Learning is a field of AI that enables systems to learn from data.

The Reward Model says:

Reward = +8

Meaning:

Good response.

Step 4: Value Model Prediction

The Value Model predicts beforehand:

Expected Reward = 6

Meaning:

V(State) = 6

Step 5: Advantage Calculation

Formula: Advantage = Actual Reward - Expected Reward

Numbers: Advantage = 8 - 6 = +2

Meaning: The response was better than expected.

Step 6: PPO Updates the Policy

Old probability: 0.60

New probability after training: 0.72

Ratio: 0.72 / 0.60 = 1.2

Meaning: The model is now more likely to generate similar helpful responses in the future.

Step 7: PPO Clipping

Suppose the probability becomes: 0.95

Ratio: 0.95 / 0.60 = 1.58

PPO says: Too much change.

Clip it.

Because PPO wants:

Small and stable updates.

What Happens After Thousands of Examples?

Example prompts:

"What is Machine Learning?"

"Explain Python."

"How does a car engine work?"

"Write a polite email."

For every prompt:

Generate Response
↓
Reward Model Scores It
↓
Value Model Predicts Expected Reward
↓
Advantage Is Computed
↓
PPO Updates the Policy

Complete RLHF Pipeline

Humans
↓
Preference Data
↓
Reward Model Training
↓
Reward Model
↓
PPO Training
↓
Policy Update
↓
Better LLM

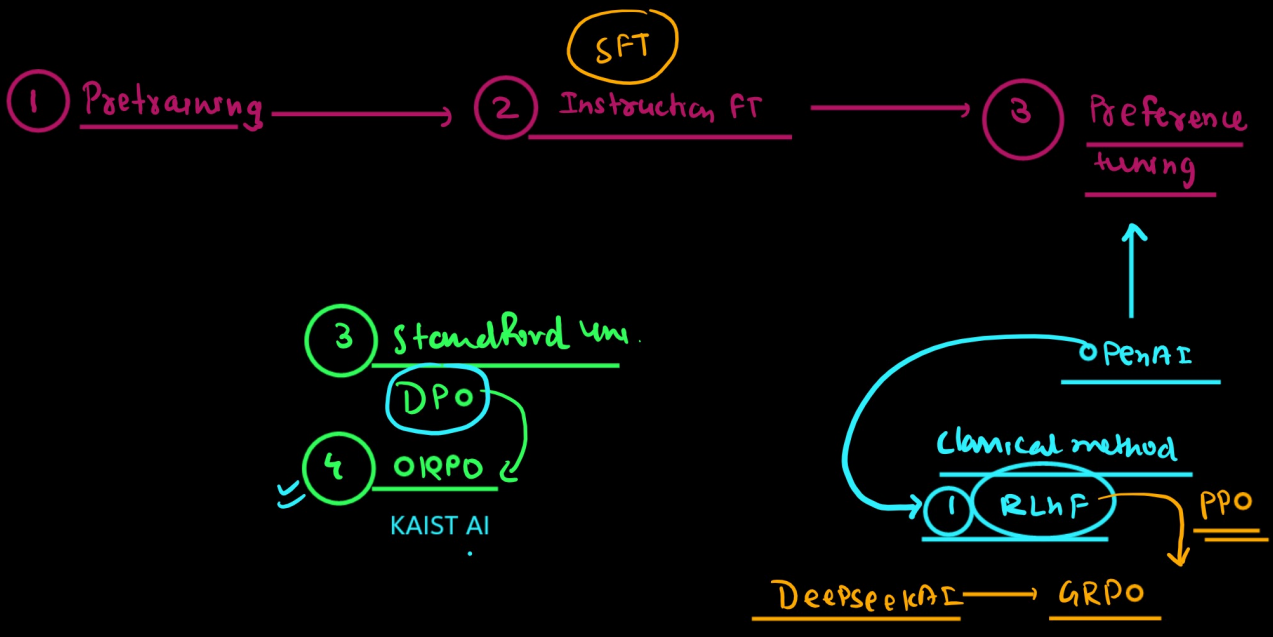
Why is Reward Modeling Needed?
An LLM does not naturally know:
• what is “better”

Policy Model = Full LLM
Reward Model = Smaller LLM + Reward Head
Value Model = Policy Model + Value Head

Who provides the reward now?
Human?
No
The Reward Model does.
Reward = 8
But who trained the Reward Model?
Human Feedback.

How an LLM Learns Human Preferences: The Story of RLHF

How do we teach the model not just to be correct, but to be preferred by humans?
That is where RLHF comes in."



What is RLHF?
RLHF is a training method or process that uses reinforcement learning algorithms like PPO, along with human feedback, to align language models(LLM) with human preferences.

SFT teach LLMs: "This is the correct answer"

RLHF teach LLMs: "This answer is better than the other one"

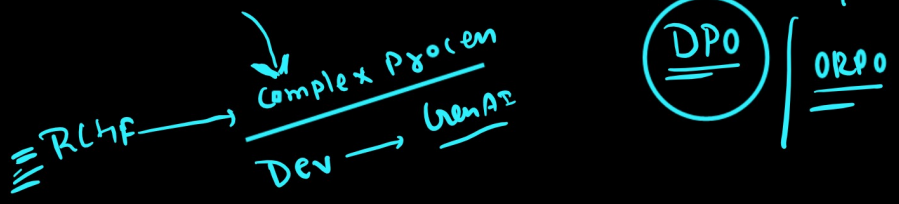
Since in the real world: There can be multiple correct answers.

Prompt: Explain AI

Output A — simple and clear:
AI means machines that can learn, think, or make decisions like humans. For example, chatbots, self-driving cars, and recommendation systems use AI.

Output B — complex and confusing:
AI is a computational paradigm involving algorithmic approximations of cognitive processes through statistical inference, representation learning, and optimization over high-dimensional data distributions.

Both can be technically correct.
but Human prefers A



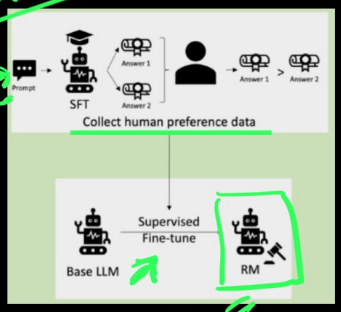
Cast of Characters

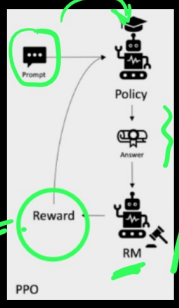
We have 5 main characters:

Human Annotators → the teachers/judges
Reward Model → the trained judge
Value Model → the coach who predicts how well the student is expected to do
Policy Model → the student
PPO → the training rule that updates the student safely"

RLHF

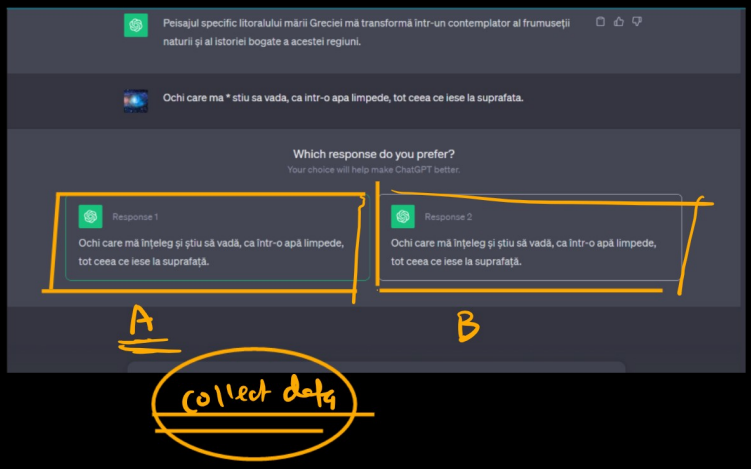

Collect human
demonstration data


Prompt
SFT
Answer 1
Answer 2
Answer 1
Answer 2
Collect human preference data
Base LLM
Supervised
Fine-tune
RM


Prompt
Policy
Answer
Reward
RM
PPO

Data Preprocessing

Step 1: Collect Human Preferences
Humans are given a prompt:
"Explain Machine Learning."
The LLM generates two responses.
Response A: "Machine Learning is a subset of AI..."
Response B: "ML is a thing that does stuff..."
The human evaluator says: "A is better than B"
We store:
Prompt
Chosen = A
Rejected = B
This is Human Feedback (HF).
and here we are preparing a data



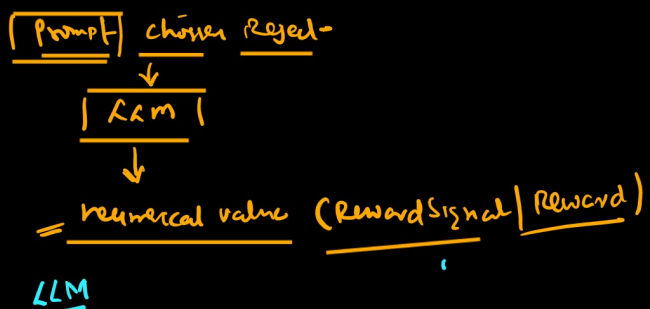
We collect this human preference as data:

Prompt
Chosen response
Rejected response

So now we are creating a preference dataset

https://huggingface.co/datasets/Anthropic/hh-rlhf

"So human feedback does not directly go into PPO.
Human feedback first trains the Reward Model.
Then the Reward Model provides the reward signal."



What is Reward Modeling?
Reward Modeling is the process of training a model to evaluate or score or rank responses according to human preferences.

In RLHF: Reward Model = A proxy for human judgment

The Reward Model acts as a proxy for human feedback. It learns human preferences from chosen/rejected examples and then approximates how a human would score a response.

Step 2: Train a Reward Model
We do not start PPO immediately.
First, we train a Reward Model.
Input:
Prompt
Response A
Response B
The Reward Model learns:
A → Reward = 8
B → Reward = 3
The Reward Model is now learning to imitate human preferences.

Reward Modeling Pipeline
Step 1: Generate multiple responses

Prompt → LLM → multiple outputs

Step 2: Human labeling (VERY IMPORTANT)
Human compares:
A > B
dataset Banega:
(prompt, chosen, rejected)

Step 3: Train Reward Model
input:
(prompt + response)

output:
score (scalar value)

RM

Objective
Reward(chosen) > Reward(rejected)
Core Idea
instead of:
correct vs incorrect
we learn:
better vs worse

Therefore:
Human
↓
Preference Data
↓
Reward Model
↓
Reward Score
↓
PPO

Step 3: PPO Begins
Policy Model (LLM)

Prompt:
"What is Machine Learning?"
The Policy Model generates a response:
"Machine Learning is a subset of AI..."
Now this response is evaluated by the
Reward Model."
Say:"Suppose the Reward Model gives:
Reward = 8
That means the answer was quite good."

Value Model(LLM)
RL needs one more thing.
It is not enough to know the actual reward.
We also want to know:

Was this better than expected, or worse than expected?
That is the role of the Value Model."

Then explain:
"The Value Model predicts the expected reward before the final judgment.
So maybe it says:

Expected Reward = 6

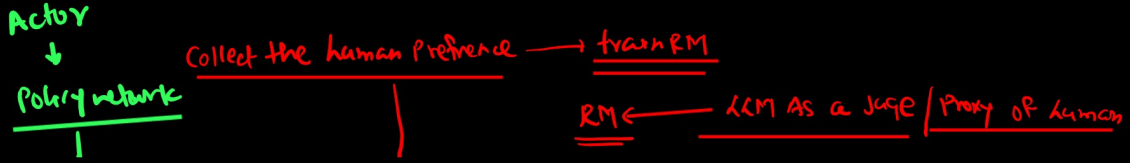
Now we compare:
Actual Reward = 8
Expected Reward = 6"

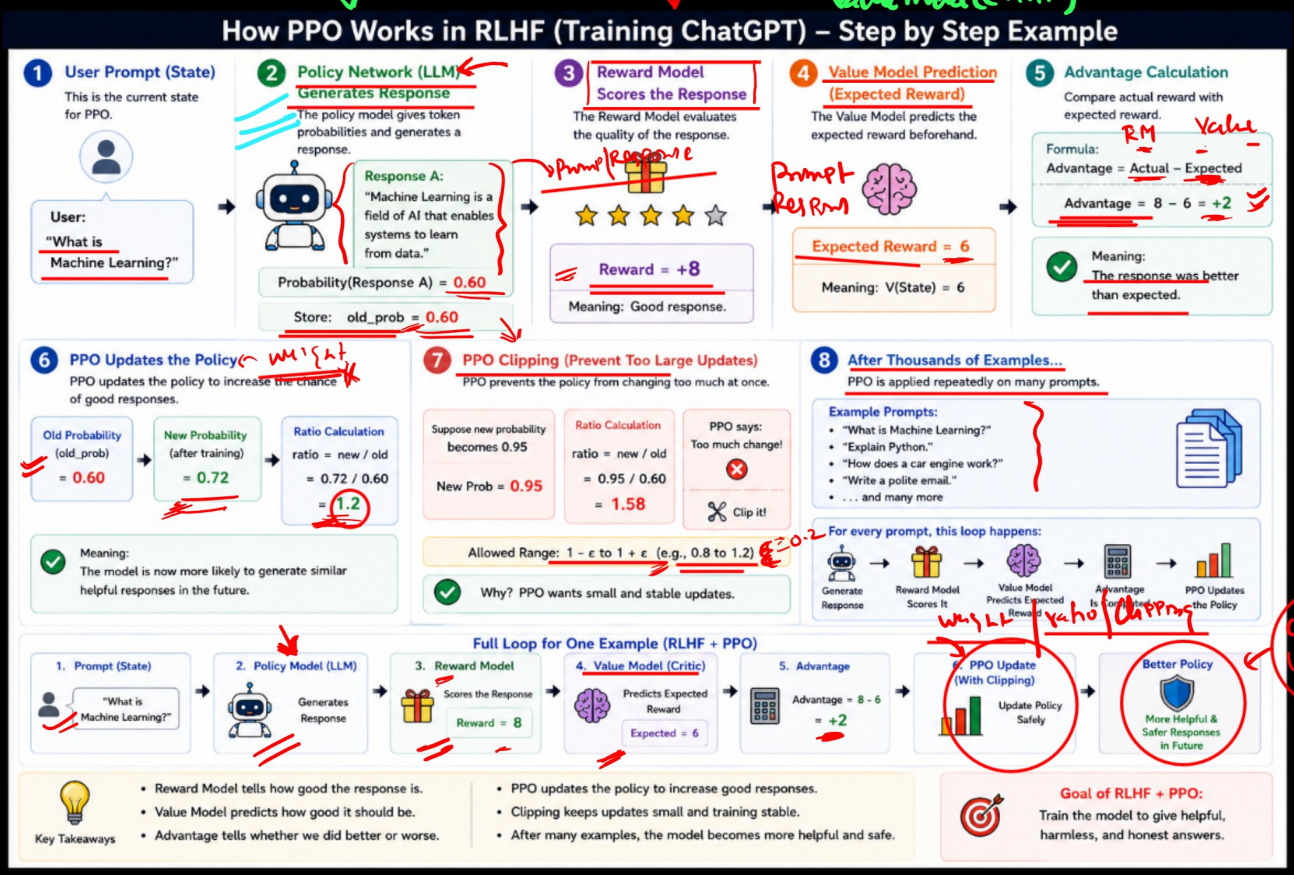
The Policy Model, Reward Model, and Value Model are usually transformer-based models. The Policy Model generates text, the Reward Model predicts a reward score, and the Value Model predicts the expected future reward used for advantage estimation.

Super Important Interview Question
Question: "Does PPO directly use Human Feedback?"
Answer: No.
Human Feedback is not directly used inside PPO.
Human Feedback is first used to train the Reward Model.
Then PPO uses the Reward Model's scores as rewards.

One-Line Summary: The "HF" in RLHF is used to train the Reward Model. PPO never interacts with humans directly; it only receives reward scores from a Reward Model that has learned human preferences.







"PPO updates the policy" means PPO modifies the model's neural network weights so that actions (or responses) that received higher rewards become more likely in the future, while lower-quality actions become less likely.

PPO Clipping
Suppose epsilon: ε = 0.2
Allowed range: 0.8 to 1.2
PPO internally says: 1.58 → Do not use it.
Instead: min(1.58, 1.2)
Use: 1.2

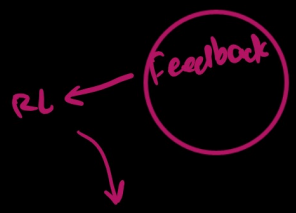
PPO clipping does not stop the neural network weights from being updated. Instead, the loss function limits the gradient so that the weight update (and therefore the probability change) cannot become too large in a single training step.

What Happens During the Weight Update?
This is a very important point. PPO does NOT directly do:
0.95 → 0.72
Instead, PPO designs the loss function in such a way that once:
ratio > 1.2
the model stops receiving additional benefit from increasing the probability further.
As a result: gradient becomes smaller
then: weight updates become smaller
and naturally: the new probability does not jump too much.

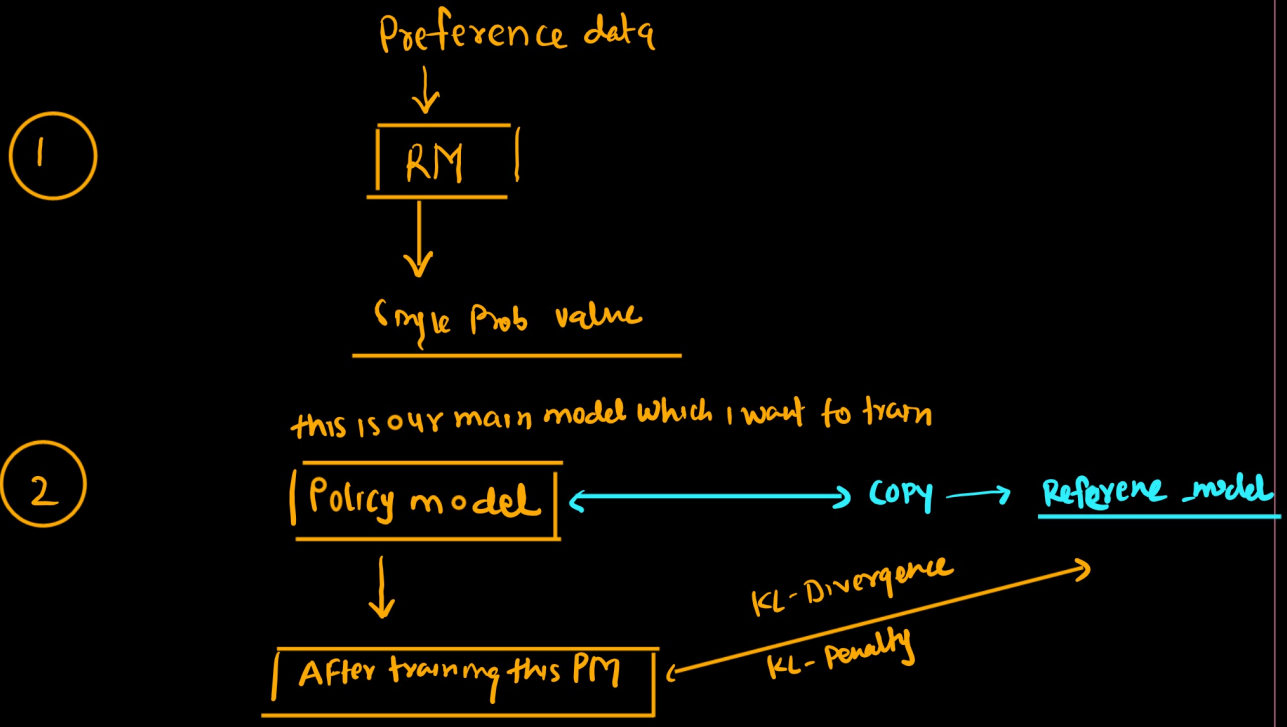
Real Intuition
Without Clipping:
0.60
↓
0.95
Huge jump.

With PPO Clipping:

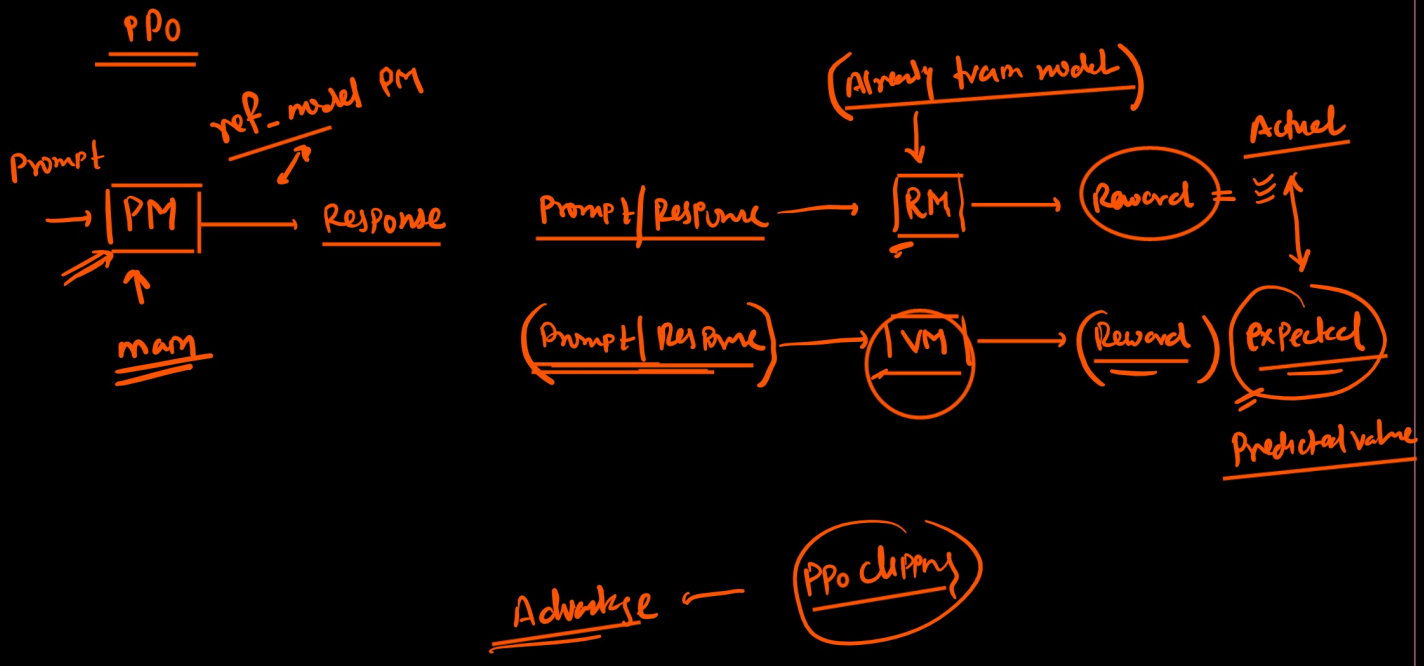
0.60
↓
0.68
↓
0.72
↓
0.75
Gradual learning.

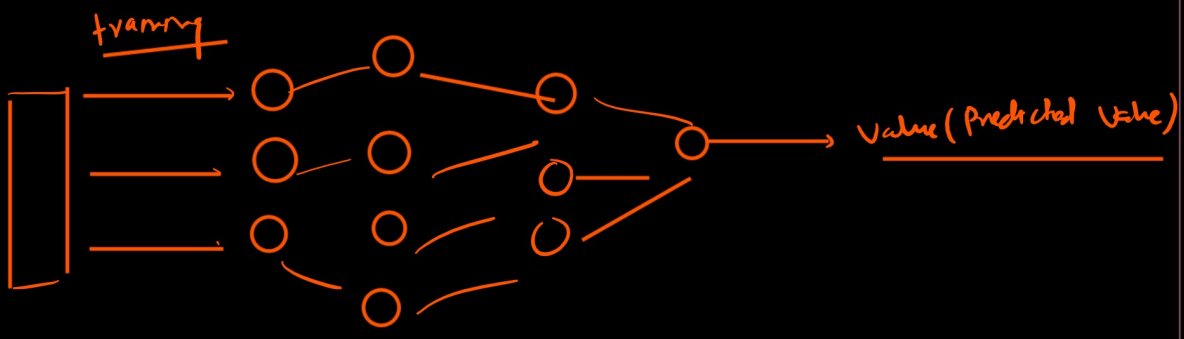


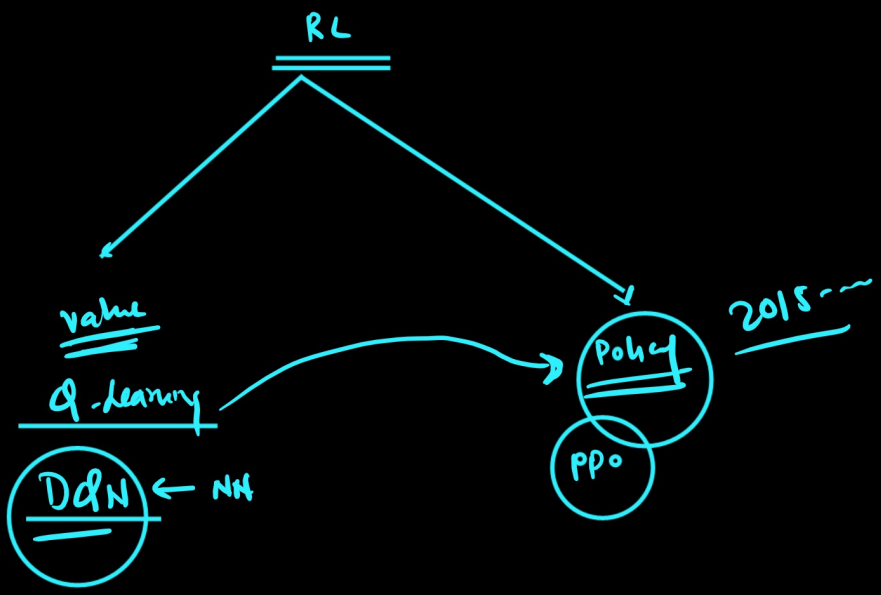
PPO is not a supervised learning because it does not learn from ground-truth labels; it learns from reward signals generated by the environment or a reward model.

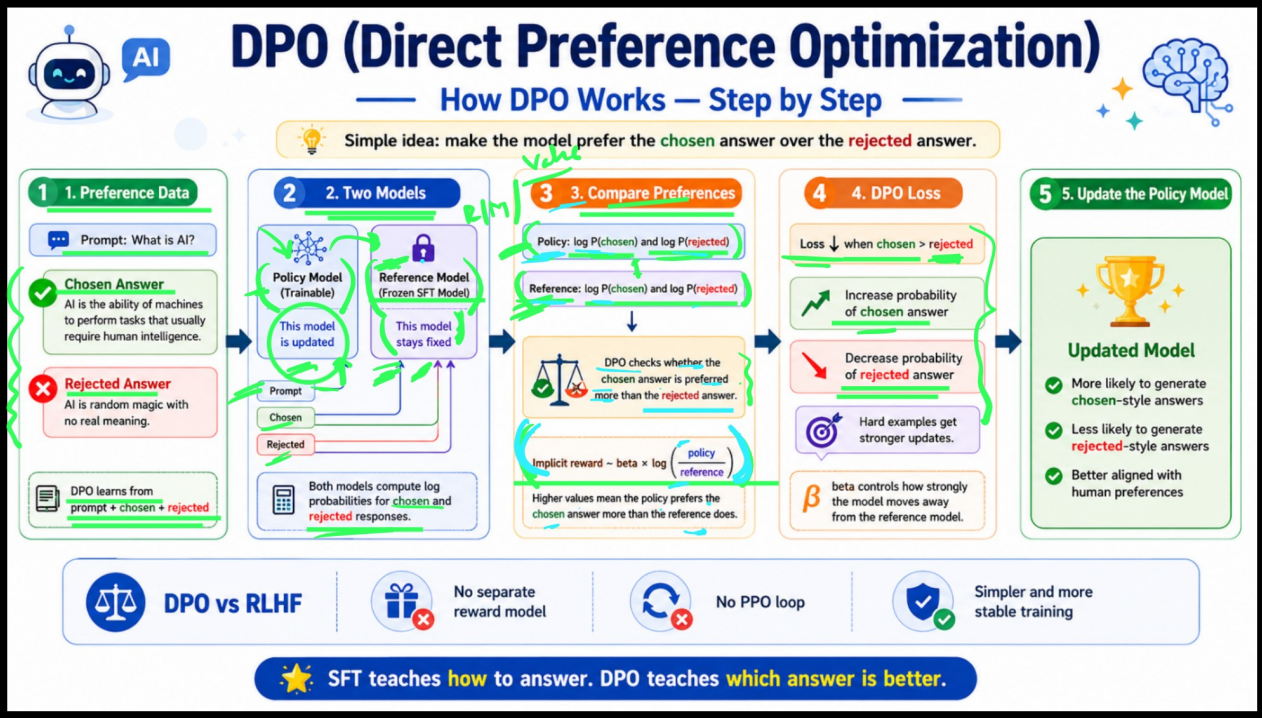

Preference data
1
RM
Single Prob value
this is our main model which I want to train
2
Policy model
COPY
Reference model
After training this PM
KL-Divergence
KL-penalty

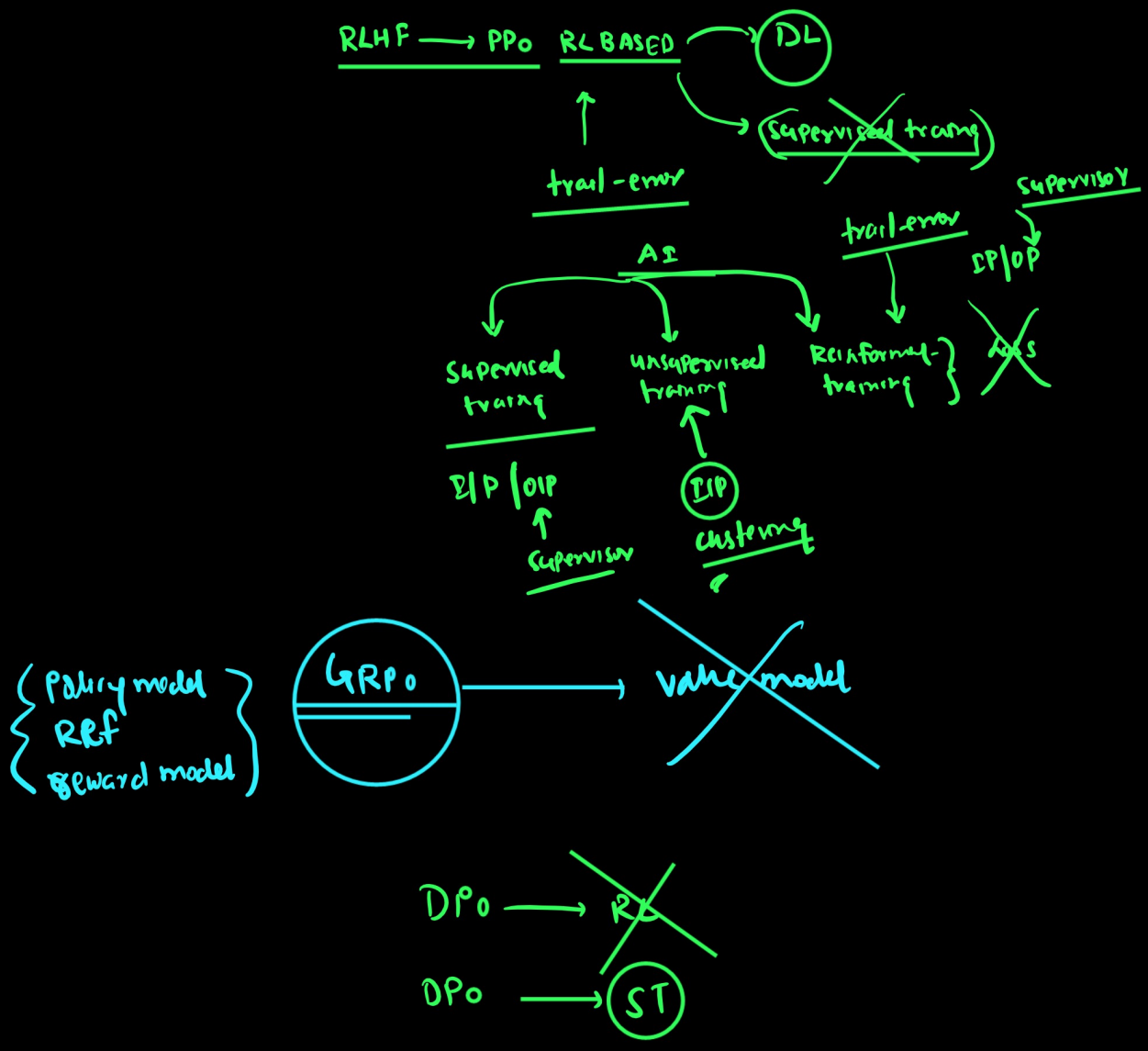

3
Value model
VM


PPO
ref_model PM
Prompt
PM
Response
main
Already train model
Prompt/Response
RM
Reward
Actual
Prompt/Response
VM
Reward
Expected
Predicted value
Advantage
PPO clipping


training
Value (Predicted Value)


RL
value
Q-learning
DQN
NN
Policy
PPO
2015~~


DPO (Direct Preference Optimization)
How DPO Works — Step by Step
Simple idea: make the model prefer the chosen answer over the rejected answer.
1. Preference Data
Prompt: What is AI?
Chosen Answer
AI is the ability of machines to perform tasks that usually require human intelligence.
Rejected Answer
AI is random magic with no real meaning.
DPO learns from prompt + chosen + rejected
2. Two Models
Policy Model (Trainable)
This model is updated
Reference Model (Frozen SFT Model)
This model stays fixed
Prompt
Chosen
Rejected
Both models compute log probabilities for chosen and rejected responses.
3. Compare Preferences
Policy: log P(chosen) and log P(rejected)
Reference: log P(chosen) and log P(rejected)
DPO checks whether the chosen answer is preferred more than the rejected answer.
Implicit reward ~ beta × log (policy / reference)
Higher values mean the policy prefers the chosen answer more than the reference does.
4. DPO Loss
Loss ↓ when chosen > rejected
Increase probability of chosen answer
Decrease probability of rejected answer
Hard examples get stronger updates.
beta controls how strongly the model moves away from the reference model.
5. Update the Policy Model
Updated Model
More likely to generate chosen-style answers
Less likely to generate rejected-style answers
Better aligned with human preferences
DPO vs RLHF
No separate reward model
No PPO loop
Simpler and more stable training
SFT teaches how to answer. DPO teaches which answer is better.


RLHF → PPO
RL BASED
DL
trail-error
supervised training
supervisor
trail-error
AI
supervised training
unsupervised training
Reinforcement-training
supervisor
clustering
Policy model
Ref
Reward model
GRPO
value model
DPO → R
DPO → ST

DPO

1. Data ⟶ Prompt, Chosen, reject

2. model ⟶ trainable
   refmodel ⟶ frozen version of model or if could be any new diff model

